

Intro2SE - Requirements Analysis - 6

Project Name

The student team is required to complete the Software Requirements Specification (SRS) document for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Table of Contents

Objectives..... 1

1 Member Contribution Assessment 2

2 Problem Statement..... 6

3 Requirements Overview 8

4 Requirements Analysis..... 22

5 AI Usage Declaration..... 49

6 Presentation..... 50

7 Reflective Report 51

Software Requirements Specification

Objectives

This document focus on the following topics:

- ✓ Completing the Software Requirements Specification (SRS) document with the following contents:
 - Elaborate on the Problem Statement
 - Overview of Requirements (Functional and Non-Functional), Stakeholders
 - Use Case Model
 - Use Case Specifications
 - Create Prototype and Mockup Diagrams of the System Interface
- ✓ Understanding the Project Requirement Analysis document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1 Member Contribution Assessment

24127254 - Hồ Đình Trí - 25%

Task Evidence:

- Task Details:

- **AP-28: Ingestion Infrastructure & Verification Layout:** Led the requirement elicitation, architecture planning, and formal system boundary analysis for the primary candidate intake pipeline.
- **AP-35: Draft Functional Requirements & Use Case Specification for Ingestion:** Documented comprehensive step-by-step main flows, strict input boundary rules, and alternative failure scenarios for file parsing validations.
- **AP-36: Design Interactive Figma Prototypes Showing Ingestion States:** Created high-fidelity user interface layouts for the IDLE drag-and-drop landing view and the active split-screen visualization dashboard.
- **AP-37: Implement Core Ingestion Components:** Defined foundational system actions regarding byte-level file headers, MIME-type verification, and maximum 10MB payload thresholds.

- Visual Evidence:

✓ AP-28 Ingestion Infrastructure & Verification Layout	 Đinh Trí	 Đinh Trí	== Medium	DONE ✓
 AP-37 Implement Core Ingestion Components (Fil...	 Đinh Trí	 Đinh Trí	== Medium	DONE ✓
 AP-36 Design interactive Figma prototypes showi...	 Đinh Trí	 Đinh Trí	== Medium	DONE ✓
 AP-35 Draft Functional Requirements & Use Case ...	 Đinh Trí	 Đinh Trí	== Medium	DONE ✓

24127382 - Trần Nhật Hoàng - 25%

Task Evidence:

- Task Details:

- **AP-32: Elaborate on Section 2: Problem Statement:** Authored the comprehensive context and problem modeling, emphasizing SHRM financial losses, linear cognitive fatigue, and domain misalignment.

- **AP-29: AI Orchestration Loop & Semantic Ranking Engine:** Supervised the behavioral and system constraint definitions for the autonomous multi-agent backend network and LLM connectors.
 - **AP-38: Draft Functional Requirements & Use Case Specification for AI Ranking:** Structured detailed functional metrics for natural language keyword parsing, dynamic slider thresholds, and LLM tool execution loops.
 - **AP-39: Design Interactive Figma Prototypes Modeling AI Dashboards:** Developed functional UI mockups demonstrating the conversational recruitment assistant interface and interactive matching lists.
 - **AP-40: Implement Core Orchestrator Code:** Specified Model Context Protocol (MCP) host adapters, context-fetching tools, and embedding transformations for pgvector storage grids.
- **Visual Evidence:**

✓ AP-29 AI Orchestration Loop & Semantic Ranking E...	HT	Hoang Tran	Đinh Trí	Medium	DONE ✓
AP-40 Implement Core Orchestrator Code (MCP ...	HT	Hoang Tran	Đinh Trí	Medium	DONE ✓
AP-39 Design interactive Figma prototypes modell...	HT	Hoang Tran	Đinh Trí	Medium	DONE ✓
AP-38 Draft Functional Requirements & Use Case ...	HT	Hoang Tran	Đinh Trí	Medium	DONE ✓
AP-32 Elaborate on the Section 2: Problem Statem...	HT	Hoang Tran	Đinh Trí	Medium	DONE ✓

24127252 - Nguyễn Khánh Toàn - 25%

Task Evidence:

- **Task Details:**
 - **AP-33: Author Section 3.1: Stakeholders Matrix:** Formulated the abstract System Context Diagram for boundary control and built the integrated Stakeholders Matrix.
 - **AP-30: Cross-Channel Profile Enrichment Engine:** Directed the architectural and event-driven analysis for the asynchronous external data harvesting pipelines.
 - **AP-41: Draft Functional Requirements & Use Case Specification for Enrichment:** Detailed functional logic specs for crawling public repositories on GitHub and professional experience timelines from LinkedIn APIs.
 - **AP-42: Design Interactive Figma Prototypes Showing Enrichment Layouts:** Created high-fidelity layouts visualizing external repository summaries, programming language ratios, and cross-platform sync signals.

- **AP-43: Implement Core Enrichment Integration:** Defined exact structural payload definitions and message synchronization rules for async events published to the Azure Service Bus queue cluster.

- **Visual Evidence:**

✓ AP-30 Cross-Channel Profile Enrichment Engine	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓
AP-43 Implement Core Enrichment Integration (A...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓
AP-42 Design interactive Figma prototypes showi...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓
AP-41 : Draft Functional Requirements & Use Case...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓
AP-33 Author the Section 3.1: Stakeholders Matrix...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓

24127337 - Nguyễn Tiến Cường - 25%

Task Evidence:

- **Task Details:**

- **AP-34: Define Non-Functional Requirements Specification:** Formulated and analyzed strict quantitative KPIs covering latency limits, encryption standards, cross-browser portability, and system scalability metrics.
- **AP-27: Contextual Scheduling & Enterprise ABAC Security:** Led the core technical analysis regarding data security, dynamic PII access policies, and automated candidate interview loops.
- **AP-44: Draft Functional Requirements & Use Case Specification for Security & Scheduling:** Drafted engineering requirements for Attribute-Based Access Control filters and Sweep-Line schedule interval overlap computations.
- **AP-45: Design Interactive Figma Prototypes Displaying Restricted Views:** Designed mockups rendering masked fields for authorized technical interviewers and step-by-step calendar integration screens.
- **AP-46: Implement Core Security & Schedule Code:** Detailed specific cryptographic token validation rules, conditional recursive JSON filtering, and Google Calendar event dispatching endpoints.

- **Visual Evidence:**

✓ AP-27 Contextual Scheduling & Enterprise ABAC Se...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓
AP-46 Implement Core Security & Schedule Code...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓
AP-45 Design interactive Figma prototypes displa...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓
AP-44 Draft Functional Requirements & Use Case ...	Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓

[AP-34](#) Define Non-Functional Requirements Speci...

CT Cường Nguyễn Tiến

Đinh Trí

= Medium

DONE ✓

2

Problem Statement

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127337 - Nguyễn Tiến Cường

2.1 Context & The Problem

In the modern corporate landscape, companies fundamentally rely on high-performing teams. To meet hiring demands, HR departments must process thousands of resumes annually. In traditional setups, this initial screening is predominantly manual, creating a slow, administrative bottleneck. This manual workflow leads to severe consequences across three main dimensions:

- **Financial Loss:** According to SHRM, the average "Time-to-Hire" spans 36–42 days. Recruiters waste hours scanning irrelevant resumes (direct labor costs), while critical roles remain vacant, delaying product launches and bleeding potential revenue (opportunity costs).
- **The "First Come, First Served" Dilemma:** Due to cognitive fatigue, recruiters process CVs linearly. Top talent applying late may sit at the bottom of a massive stack and never be opened, making candidate selection luck-based rather than merit-based.
- **HR & Technical Misalignment:** Manual screening is subjective and prone to human bias. Furthermore, generalist recruiters often judge CVs based on superficial buzzwords. This domain-knowledge gap leads to mismatched candidates, wasted interview rounds, and high turnover rates.

2.2 The Proposed Solution

To resolve these bottlenecks, we propose an **AI-powered Applicant Tracking System (ATS)**. Instead of rigid keyword matching, the system leverages **Large Language Models (LLMs)** and **Semantic Search** to comprehend and rank resumes based on contextual capability.

The core innovation is the integration of autonomous **AI Agents** via the **Model Context Protocol (MCP)**. This architecture allows specialized agents (e.g., HR Agent and Technical Agent) to securely fetch data and collaborate on resume summaries in real-time. This eliminates human bias, slashes Time-to-Hire, and bridges the gap between HR evaluation and technical reality.

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

2.3 Operating Environment

The AI-powered ATS is deployed as a cloud-native, client-server web application operating in the following environment:

- **Client-Side:** Responsive web interface accessible via modern browsers supporting HTML5, CSS3, and ES6+ (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari). Fully optimized for Desktop and Tablet viewports.
- **Server-Side:** Containerized environments deployed via Docker-Compose for development, targeting distributed cloud infrastructure.
- **AI Engine:** Secure API connectivity to LLM providers (e.g., Hugging face) integrated with an embedded **MCP Host** layer to orchestrate live agent-to-data communication.

2.4 Design & Implementation Constraints

Tech Stack Constraints

- **Backend:** Python 3.11+ with **FastAPI** for high-performance API routing and native integration with LLM and MCP SDKs.
- **Frontend:** **React.js (TypeScript)** and **Tailwind CSS** for a dynamic, component-driven user interface.
- **Database:** **PostgreSQL** with the **pgvector** extension to handle both structured relational data and vector embeddings for semantic search.
- **AI Integration:** **LangChain** (or **LlamaIndex**) for agent orchestration, and the official **MCP SDK** for secure protocol connectors.

Design Constraints

- **Architectural Pattern:** Strict **Client-Server Separation** via RESTful APIs. The AI execution layer must be decoupled from the core business logic to prevent interface lag during heavy AI inference.

- **Data Privacy & Security:** All uploaded CVs must be encrypted at rest using **AES-256**. User authentication and endpoint protection must strictly comply with **OAuth 2.0** and **JWT** standards to protect sensitive personal data (PII).
- **User Interface (UI) Design:** Built upon **Material Design** guidelines, prioritizing high-contrast typography for resume scanning and data-dense dashboards for recruitment metrics.

Documentation Standards

- **System Modeling:** Architecture must be documented using **UML 2.0 diagrams**, including Use Case, Class (database schema), and Sequence diagrams detailing the recruiter-to-agent workflow.
- **Code Documentation:** Source code must follow the **Google Python Style Guide (PEP 8)**, enforcing strict Type Hinting and clear docstrings for all public APIs and AI tools.

3 Requirements Overview

3.1 Stakeholders

Written by: 24127252 - Nguyễn Khánh Toàn

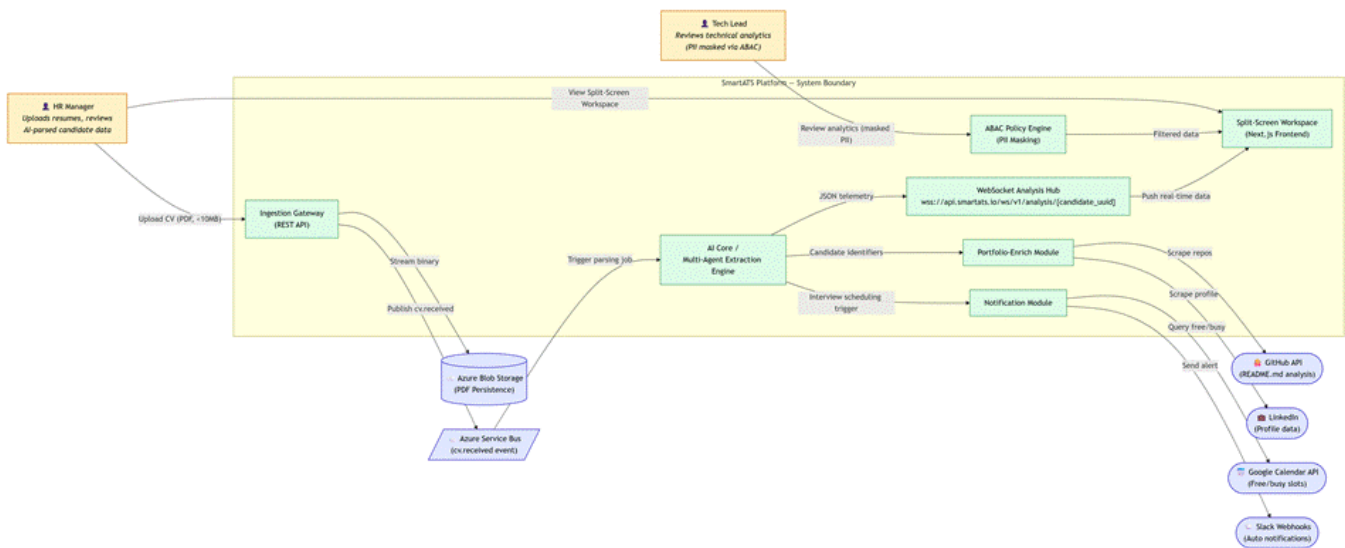
Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường

3.1.1 System Context Diagram

The diagram below positions SmartATS as the central system under design, bounded by a single dashed-line boundary that separates internal components from external actors and third-party systems. Two human actor roles sit on the left of the boundary: the HR Manager, who drives the core ingestion and review workflow, and the Tech Lead, who consumes analytical output under an ABAC-restricted, PII-masked view.

On the right, the diagram maps every external system the platform integrates with: Azure Blob Storage and Azure Service Bus form the asynchronous ingestion backbone; GitHub and LinkedIn are scraped by the Portfolio-Enrich module to augment candidate profiles; Google Calendar API is queried for interviewer availability; and Slack Webhooks deliver automated notifications back to the hiring team. The bidirectional WebSocket channel (`wss://api.smartats.io/ws/v1/analysis/{candidate_uuid}`) is shown explicitly, since it is the primary real-time data path between the AI Core and the Split-Screen Workspace UI. This diagram establishes the system's outer boundary and integration surface, serving as the baseline reference for all subsequent Use Case and Sequence diagrams in this document.



3.1.2 Stakeholders Matrix

STT	Stakeholder	Type	Description
1	Project Sponsor	Internal / Business	Funds and oversees the SmartATS initiative; approves scope, budget, and final delivery acceptance.
2	HR Manager	Primary Human Actor	Uploads candidate resumes, monitors ingestion status, and reviews AI-parsed analytical output via the Split-Screen Workspace. Holds full visibility into candidate PII.
3	Tech Lead	Secondary Human Actor	Reviews technical-skill analytics (Radar Chart, career timeline) for hiring decisions; operates under ABAC policy with PII fields masked/redacted.
4	Candidate (Data Subject)	Indirect Stakeholder	Submits CV data indirectly through the recruitment process; does not interact with the system directly but is the subject of all processed data, governed by data-privacy obligations.

5	Autonomous AI Analytics Engine (AI Core)	Internal System Component	Multi-agent backend service that parses ingested PDFs, extracts structured candidate metadata, and streams results to the frontend via WebSocket.
6	Azure Blob Storage	External System	Cloud persistence layer storing raw candidate PDF binaries under a private container, referenced by a deterministic UUID-mapped URL.
7	Azure Service Bus	External System	Asynchronous message broker decoupling the ingestion API from the downstream AI parsing pipeline via the <code>cv.received</code> event.
8	GitHub	External Third-Party System	Source queried by the Portfolio-Enrich module to analyze public repositories and README.md content for technical-skill corroboration.
9	LinkedIn	External Third-Party System	Source queried by the Portfolio-Enrich module to enrich candidate profiles with professional history data.
10	Google Calendar API	External Third-Party System	Queried by the Notification module to identify interviewer free/busy slots for scheduling.
11	Slack Webhooks	External Third-Party System	Outbound channel used by the Notification module to push automated hiring-pipeline alerts to internal HR/recruiting channels.
12	System Administrator	Internal Operational Actor	Manages ABAC policy configuration, system uptime, and infrastructure scaling (Azure resources, message queue throughput).

3.2 Requirements

3.2.1. Functional Requirements Specification

a/Ingestion Infrastructure & Verification Layout

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

-Multipart File Ingestion Gateway: **+Description:** The system must provide a secure **RESTful API** endpoint accepting form-data payloads to allow authorized HR Managers to upload raw candidate resume documents.

+System Action: Intercept the payload at the API gateway, validate that the **MIME type** strictly matches **application/pdf**, and enforce a hard boundary limits check where files exceeding **10MB** are immediately blocked

-Deterministic Storage and Cloud Persistence:

+Description: Validated assets must be safely structured and stored in isolated cloud environments to eliminate data fragmentation risks.

+System Action: Stream the raw binary PDF data stream directly to **Microsoft Azure Blob Storage** under a secure private container registry, generating a deterministic, non-guessable storage URL mapped to a fresh candidate Universally Unique Identifier (UUID).

-Asynchronous Event Broker Broadcast:

+Description: The system must decouple the heavy processing pipeline from the client thread using an event-driven mechanism to guarantee performance stability.

+System Action: Initialize an integration transaction and publish a structured **cv.received** event payload—containing the candidate UUID, storage URL reference, and timestamps—to the **Azure Service Bus message queue backbone**, then instantly flush an **HTTP 202 Accepted** status back to the frontend application client.

-Split-Screen Layout Initialization:

+Description: The responsive web interface must render a multi-panel workspace layout optimized for comparative verification.

+System Action: Initialize a Split-Screen Workspace flexbox container grid. The left-hand panel must pull the candidate asset from the storage URL via a client-side streaming wrapper, rendering the native PDF document layout page-by-page dynamically.

-Real-Time WebSocket Synchronization Hub:

+**Description:** The UI must establish bidirectional pipes to push backend analytical intelligence to the dashboard view without full-page reloads.

+**System Action:** Upon workspace layout mounting, open a secure persistent WebSocket sub-channel (wss://api.smartats.io/ws/v1/analysis/{candidate_uuid}). The frontend client must transition into a listening state, awaiting data frames from the downstream multi-agent extraction loop nodes.

-Interactive Visual Analytics Projection:

+**Description:** The system must map complex semantic metrics into clear, scannable visual components once background processing finishes.

+**System Action:** Consume incoming WebSocket JSON metadata payloads containing structured candidate details. Render an interactive, high-dimensional **Canvas Radar Chart** on the right-hand layout panel to visualize technical skill distributions, and project an expandable SVG career timeline summarizing verified employment history alongside the original document.

[Describe the functional requirements of the system using natural language. The combination of forms and regulations is acceptable. **Number and group the requirements accordingly**]

b/ Access Control & Interview Scheduling

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

-ABAC PII Data Masking for Technical Interviewer

+**Description:** When a Technical Interviewer (role) sends a GET request to retrieve a candidate's detailed profile, the system **MUST** apply a recursive masking function that overwrites sensitive PII fields (email, phone_number, expected_salary) with the string '***' before returning the JSON response. This ensures compliance with data privacy regulations and prevents unconscious salary bias during technical evaluation.

+**System Action:** Intercept the incoming API call at the Gateway. Decode the JWT and extract the role claim. If the role equals 'interviewer', invoke the ABAC Policy Engine which iterates over the candidate JSON payload recursively, overwriting the values of email, phone_number, and expected_salary fields with the string '***'. Return the sanitized JSON response with a HTTP 200 OK status.

- Contextual Interview Scheduling via Google Calendar Sweep-Line Algorithm

+**Description:** The system MUST connect to the Google Calendar API to fetch discrete free/busy time blocks for all selected Tech Lead interviewers. It MUST apply a Sweep-Line overlap detection algorithm to identify common free time slots where every participant is available. Each proposed slot MUST have a minimum duration of 45 minutes. The ordered list of valid slots is rendered in the UI for the HR Manager to select and confirm.

+**System Action:** Accept the candidate UUID and list of Tech Lead interviewers from the frontend request. Query each Tech Lead's Google Calendar via the free/busy API endpoint for the next 14 days. Collect all busy intervals, invert them to obtain free intervals per individual, then apply the Sweep-Line algorithm: sort all interval endpoints chronologically, sweep with a running counter, and extract contiguous ranges where the counter equals the total number of interviewers. Filter ranges shorter than 45 minutes. Return the sorted list of valid slots as a JSON array.

- Automated Interview Confirmation Notification

+ **Description:** Once the HR Manager confirms an interview slot, the system MUST create a Google Calendar event with the candidate and selected interviewers as attendees, and dispatch a structured notification payload to the configured Slack Webhook channel indicating the interview time, candidate name, and role.

+ **System Action:** Upon HR Manager slot confirmation, call the Google Calendar Events API to create a calendar event with the candidate name, interviewers as attendees, and the selected time slot. Simultaneously, POST a structured JSON payload to the configured Slack Webhook URL containing candidate name, role, interview time, and a deep-link to the candidate profile. Return HTTP 201 Created to the frontend with the calendar event ID.

c) Candidate Ingestion & AI Processing

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- Secure CV Submission

+ **Description:** The system shall provide a secure web interface allowing candidates to upload resume documents in PDF format.

+ **System Action:** Validate the uploaded file, generate a unique Candidate UUID, and store the document before initiating the AI processing pipeline.

- Automated CV Analysis Pipeline

+ **Description:** The system shall automatically transform raw resume documents into structured candidate profiles.

- + **System Action:** Parse the uploaded CV, generate an LLM-based candidate summary, create semantic embeddings, and persist all processed data into the candidate database.

d) Semantic Candidate Ranking

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- Intelligent Candidate Search

- + **Description:** HR users shall be able to retrieve candidates using natural-language job requirements.
- + **System Action:** Generate semantic embeddings for the job requirement, perform vector similarity search, calculate semantic matching scores, and return candidates ranked by relevance.

- Dynamic Matching Threshold

- + **Description:** HR users shall be able to dynamically adjust the minimum semantic similarity score.
- + **System Action:** Recompute and refresh the ranked candidate list whenever the threshold slider value changes.

e) Autonomous LLM Orchestration

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- AI Workflow Orchestration

- + **Description:** The system shall automatically determine the appropriate workflow based on user intent.
- + **System Action:** Analyze user requests, invoke the appropriate AI tool (Semantic Ranking, Interview Scheduling, Candidate Lookup, etc.), and aggregate responses before returning results.

- AI Recruitment Assistant

- + **Description:** Users shall communicate with the system through a conversational AI interface.
- + **System Action:** Receive natural-language requests, forward them to the LLM Orchestrator, execute the selected tools, and generate contextual responses.

f/ Cross-Channel Profile Enrichment Engine

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường

External Profile Identifier Extraction:

- Description: Upon completion of the CV ingestion phase, the system must automatically locate and extract all recognizable external profile identifiers embedded within the candidate's parsed document record.
- System Action: Scan the structured candidate JSON payload produced by the upstream parsing worker to detect and capture any GitHub profile URL and LinkedIn profile URL fields. Store the extracted identifiers as indexed attributes on the candidate record within the PostgreSQL database before dispatching enrichment tasks.

Asynchronous Enrichment Pipeline Dispatch:

- Description: The enrichment pipeline must be initiated as a fully non-blocking background operation that does not interrupt the active split-screen workspace session or the primary CV parsing flow.
- System Action: Publish a dedicated profile.enrich event payload — containing the candidate UUID and the extracted external profile identifiers — to the Azure Service Bus message queue backbone. The enrichment worker pool independently consumes this event without blocking the API Gateway thread.

GitHub Repository Intelligence Extraction:

- Description: The system must crawl the candidate's public GitHub presence to derive objective, evidence-based signals of real-world technical capability beyond what is self-reported in the CV.
- System Action: Invoke the authenticated GitHub REST API to retrieve the candidate's public repository index. For each repository, extract the full README.md content for NLP-based project summarization, compute the programming language distribution ratio from repository metadata, and derive a contribution activity timeline from the commit history records.

LinkedIn Professional History Harvesting:

- Description: The system must retrieve structured professional history data from the candidate's LinkedIn profile to supplement and cross-validate the experience and education claims declared in the raw CV.
- System Action: Dispatch a secure outbound request to the integrated third-party LinkedIn data provider (Proxycurl API or equivalent rotating-proxy service) to harvest the candidate's structured employment experience timeline, educational background, active skill endorsements, and professional certification records.

Unified Candidate Profile Consolidation:

- Description: All enriched external metadata harvested from GitHub and LinkedIn must be normalized and merged into the candidate's existing database record under a consistent, canonical data schema.
- System Action: Apply an idempotent merge operation on the candidate's CandidateProfile JSON structure within the PostgreSQL database, integrating the enriched skill attributes, project summaries, and professional history fields without duplicating pre-existing entries from the original CV parsing output.

Semantic Re-Indexing & Smatch Score Recalculation:

- Description: The AI Analytics Engine must incorporate the newly enriched profile data to produce a recalibrated semantic match score that accurately reflects the candidate's full cross-platform capability set.
- System Action: Signal the AI Analytics Engine to re-execute vector transformation procedures on the updated candidate profile via the pgvector extension, then recalculate the Smatch score against the active Job Description embedding. Persist the revised score and updated justification log to the database.

Real-Time Enrichment Result Projection:

- Description: The HR Manager's workspace must reflect enriched analytics automatically upon pipeline completion without requiring a manual page refresh.
- System Action: Transmit the finalized enriched analytics payload through the active WebSocket channel mapped to the candidate UUID. The frontend must refresh the right-side panel to render the updated Canvas Radar Chart, revised SVG career timeline, and recalculated programming language distribution chart in-place.

Manual Sync Trigger Control:

- Description: HR Managers must retain explicit on-demand control to re-initiate the enrichment pipeline at any stage of the candidate review lifecycle.
- System Action: Expose a "Run Sync" action button on the candidate profile view. Upon activation, the system re-publishes a profile.enrich event to the message queue, resetting the enrichment status to QUEUED and restarting the full external data harvesting sequence for the targeted candidate record.

Enrichment Status Transparency:

- Description: The HR Manager must have continuous visibility into the real-time processing state of the enrichment pipeline for each candidate.
- System Action: Maintain a persisted enrichment_status field on each candidate database record, cycling through discrete states: QUEUED → IN_PROGRESS → ENRICHED /

ENRICHMENT_FAILED / NO_PROFILES_FOUND. Push status transition events to the frontend via the active WebSocket channel to update the visible status indicator on the candidate dashboard in real time.

Missing Identifier Graceful Degradation:

- Description: The system must handle candidates whose CVs contain no detectable external profile links without crashing the pipeline or corrupting the existing baseline analytics record.
- System Action: If no GitHub URL or LinkedIn URL is extractable from the candidate's parsed JSON record, update the candidate enrichment status to NO_PROFILES_FOUND, emit a warning notification frame through the WebSocket channel, and surface a descriptive banner alert on the workspace UI. All existing baseline CV analytics must remain intact and unchanged.

3.2.2. Non-Functional Requirements Specification

a/Ingestion Infrastructure & Verification Layout

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

-Low-Latency File Rendering (Performance)

+**Constraint:** The system must guarantee that the first page of the candidate's native PDF document is rendered and completely interactive inside the left viewport layout within **800 milliseconds** after the cloud storage confirmation is received.

-High-Velocity WebSocket Throughput (Performance)

+**Constraint:** Once the background multi-agent loop finishes parsing the structured candidate profile, the WebSocket channel must broadcast and transfer the complete JSON metadata payload to the right-hand panel viewport container within a maximum threshold of **200 milliseconds**.

-Responsive Grid Adaptability (Usability/Design)

+**Constraint:** The split-screen dual-panel workspace layout grid must adapt fluidly across varied desktop monitoring resolutions (minimum width of **1280px** up to **4K displays**) utilizing responsive Flexbox/Grid structures, maintaining stable view proportions without horizontal text-overflow clipping.

-Concurrent Ingestion Load Capacity (Scalability)

+**Constraint:** The message queue architecture (Azure Service Bus) coupled with the stateless API gateway must support an ingestion load capacity handling up to **100 simultaneous multi-part file uploads per minute** without dropped connections or message duplication.

-Input Scrubbing & Perimeter Guardrails (Security)

+**Constraint:** The Multipart File Ingestion Gateway must conduct strict byte-level header checks on all uploaded assets to prevent extension spoofing attacks, immediately terminating transactions that contain hidden malicious executable macros disguised within the PDF stream.

-Cross-Browser Engine Compatibility (Portability)

+**Constraint:** The client-side PDF canvas streaming engine and persistent WebSocket communication modules must operate seamlessly and maintain uniform styling matrices across major web browsers compliant with the **ECMAScript 2024** execution standard, including Google Chrome, Mozilla Firefox, and Apple Safari.

b/ Access Control & Interview Scheduling

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

-ABAC Masking Response Time (Performance)

+**Constraint:** The ABAC Policy Engine **MUST** complete the recursive PII masking operation and return the sanitized JSON payload within 150 milliseconds after the Gateway receives the request. This threshold applies to candidate profiles containing up to 50 fields total.

-PII Field Coverage Ratio (Security)

+**Constraint:** The masking function **MUST** cover 100% of all PII fields defined in the ABAC policy configuration (email, phone_number, expected_salary minimum). Any field added to the policy configuration **MUST** be automatically detected and masked without requiring a backend deployment.

-Calendar API Query Deadline (Performance)

+**Constraint:** The Scheduling Engine **MUST** complete the Google Calendar free/busy queries for up to 5 Tech Lead interviewers and execute the Sweep-Line slot computation within 3 seconds. If the API response exceeds this threshold, the engine **MUST** return a partial result with a warning header indicating incomplete data.

-Slot Computation Precision (Reliability)

+Constraint: The Sweep-Line algorithm MUST identify overlapping free slots with millisecond precision. The algorithm MUST guarantee zero false-positive overlaps (slots where any interviewer is actually busy) and MUST NOT propose slots shorter than 45 minutes.

-Calendar Service Degradation Handling (Availability)

+Constraint: If the Google Calendar API is unreachable or returns HTTP 5xx errors, the Scheduling Engine MUST fall back to cached free/busy data (if available) and display a warning banner in the UI. If no cached data exists, the engine MUST return an empty slot list with an explicit availability error message within 2 seconds.

-Notification Delivery Deadline (Performance)

+Constraint: The system MUST create the Google Calendar event AND dispatch the Slack Webhook notification within 1 second of the HR Manager confirming the interview slot. If either channel fails, the system MUST retry once after 500ms; if both attempts fail, the error MUST be logged and a warning banner displayed in the UI.

c) Candidate Ingestion & AI Processing

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- Secure File Validation (Security)

- + **Constraint:**The system shall validate uploaded CV files by checking MIME type, file extension, and maximum file size before processing. Unsupported or potentially malicious files shall be rejected immediately.-

- Processing Performance (Performance)

- + **Constraint:**The complete CV processing pipeline, including parsing, AI summary generation, and embedding creation, should complete within **10 seconds** under normal operating conditions.

- Reliable Document Storage (Reliability)

- + **Constraint:**Uploaded CV documents shall be securely stored with unique identifiers, ensuring no data loss during processing or storage failures.

d) Semantic Candidate Ranking

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- **Low-Latency Candidate Retrieval (Performance)**
 - + **Constraint:**Semantic candidate ranking should return ranked results within **3 seconds** for a candidate database containing up to **10,000 profiles**.
- **Dynamic Ranking Responsiveness (Usability)**
 - + **Constraint:**Adjusting the semantic matching threshold slider shall refresh candidate rankings with minimal delay, providing a smooth interactive experience.
- **Ranking Accuracy (Quality)**
 - + **Constraint:**The ranking algorithm shall consistently retrieve candidates based on semantic similarity rather than keyword matching alone.

e) Autonomous LLM Orchestration

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

- **AI Response Time (Performance)**
 - + **Constraint:**The AI Assistant should provide responses within **5 seconds**, excluding delays introduced by external LLM service providers.
- **Modular Tool Integration (Maintainability)**
 - + **Constraint:**The Autonomous LLM Orchestrator shall support the addition of new AI tools without requiring significant modifications to existing orchestration workflows.
- **Fault Tolerance (Reliability)**
 - + **Constraint:**Failures of individual AI tools shall not interrupt the overall system. The orchestrator shall gracefully handle tool failures and provide informative responses to users.
- **Secure AI Communication (Security)**
 - + **Constraint:**All communications between the AI Orchestrator and external AI services shall be encrypted using secure HTTPS/TLS protocols.

f/ Cross-Channel Profile Enrichment Engine

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường

- **Non-Blocking Async Compliance (Performance)**
 - o Constraint: The enrichment pipeline dispatch must publish the profile.enrich event to the Azure Service Bus queue within 500 milliseconds of the upstream CV parsing completion signal. Throughout the entire background enrichment execution, the primary HR workspace UI must maintain a client-side interaction response time below 200 milliseconds to preserve workspace usability.
- **End-to-End Enrichment Throughput (Performance)**
 - o Constraint: The complete enrichment cycle — spanning external API crawling, LinkedIn harvesting, profile consolidation, vector re-indexing, and Smatch recalculation — must deliver the finalized updated analytics payload to the client WebSocket channel within 45 seconds under standard network conditions and nominal third-party API response latencies.
- **Fault Tolerance & Retry Resilience (Reliability)**
 - o Constraint: The enrichment integration layer must enforce an Exponential Backoff retry policy across all outbound external API calls, executing a minimum of 3 retry attempts with progressively increasing backoff intervals before declaring a hard failure. Upon exhausting all retry attempts, the system must transition the candidate enrichment status to ENRICHMENT_FAILED, emit a fallback alert via the active WebSocket channel, and guarantee zero corruption of the existing baseline candidate analytics record.
- **Third-Party Rate Limit Compliance (Reliability)**
 - o Constraint: The enrichment engine must enforce internal request throttling logic to strictly comply with external provider rate-limit policies, including the GitHub REST API authenticated ceiling of 5,000 requests per hour. The throttling layer must prevent API token revocations, temporary IP bans, or service suspensions from any integrated third-party data provider.
- **Enrichment Worker Scalability (Scalability)**
 - o Constraint: The enrichment worker pool must support horizontal scaling via the Azure Service Bus message queue backbone. Under peak load conditions — defined as 50 or more simultaneous enrichment events queued concurrently — all tasks must be processed to completion without degrading API Gateway response times for active HR Manager sessions.
- **Credential Isolation & Secret Hygiene (Security)**
 - o Constraint: All outbound authentication credentials used for third-party API integrations — including GitHub API tokens and LinkedIn provider API keys — must be stored exclusively within server-side environment-scoped secret vaults. No authentication tokens may be exposed to the client application layer, embedded in frontend bundles, or recorded in plaintext within the system's centralized audit log.

- **Idempotent Data Merge Integrity (Data Integrity)**
 - o Constraint: The profile consolidation operation must be fully idempotent. Re-executing the enrichment pipeline against the same candidate UUID any number of times must consistently produce an identical final database state, with no duplication of skill entries, employment records, or repository data within the PostgreSQL candidate profile schema.
- **Enrichment Auditability (Maintainability)**
 - o Constraint: Every enrichment pipeline execution must emit a structured audit log entry — persisted to the system's centralized audit trail — recording the trigger timestamp, external API response status codes, retry attempt counts, and final pipeline outcome, all traceable by candidate_uuid for retrospective debugging and compliance verification.
- **Module-Level Deployment Independence (Maintainability)**
 - o Constraint: The Portfolio-Enrich Engine must be implemented and deployable as an independently isolated module within the Modular Monolith architecture, decoupled from both the CV Parser and AI Analytics Engine exclusively through the message queue boundary. Upgrades or full replacements of any external data provider integration must require zero modifications to the API Gateway layer or the AI Analytics Engine codebase.

4

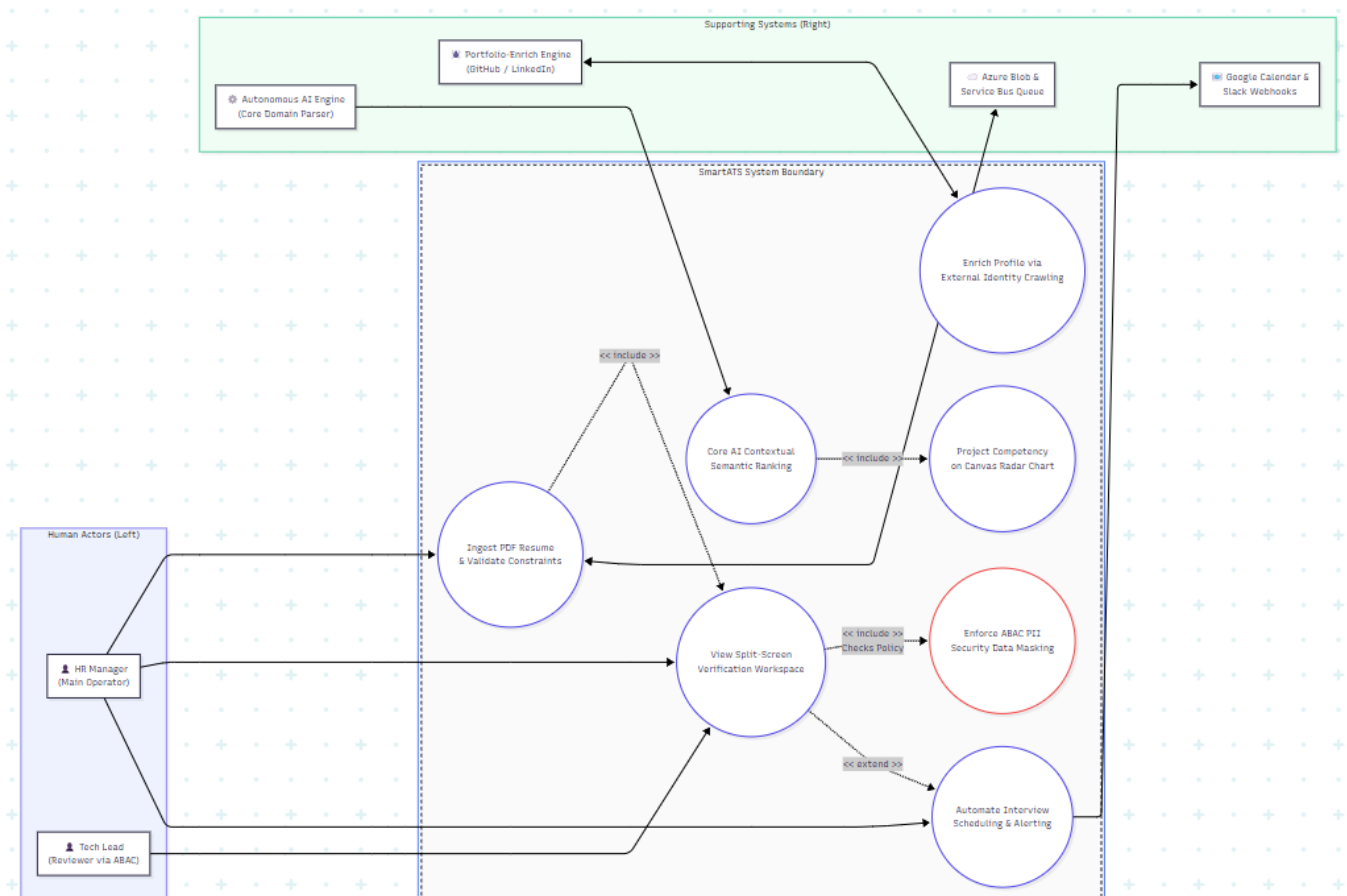
Requirements Analysis

4.1 Use Case model

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127382 - Trần Nhật Hoàng



System Boundary and Actor Interaction Overview: The Use Case Diagram above establishes the operational boundary of the SmartATS platform, illustrating the behavioral relationships between external actors and internal system components. The architecture is designed to decouple user-facing interactions from automated background processes to ensure system stability and high performance.

- Primary Human Actors:** The **HR Manager** serves as the primary system operator, holding explicit privileges to initiate the core file ingestion pipeline, view the synchronized split-screen verification workspace, and manage interview schedules. Conversely, the **Tech Lead** acts as a secondary reviewer, strictly bounded by Attribute-Based Access Control (ABAC) policies that dynamically redact and mask Candidate Personally Identifiable Information (PII) to ensure data privacy.
- Core System Use Cases & Internal Relationships: Ingestion Layer:** When a resume is uploaded, the system validates file constraints (MIME types and size boundaries under 10MB), which automatically triggers (<<include>>) the mounting of the Split-Screen Verification Workspace.

AI Analytics Layer: Initiated by the **Autonomous AI Engine**, the system extracts contextual metadata and calculates semantic proximity scores, which are immediately

projected onto an interactive Canvas Radar Chart (<<include>>) via a live WebSocket stream.

Profile Enrichment: Operating asynchronously, the system invokes the **Portfolio-Enrich Engine** to crawl external GitHub and LinkedIn profiles, augmenting the candidate record without interrupting the client thread.

Security & Automation: The verification layout strictly incorporates (<<include>>) ABAC security protocols to mask data fields before rendering analytics to non-authorized roles. Furthermore, the workspace dynamically extends (<<extend>>) its capabilities to allow direct interview scheduling and automated alerting via third-party channels (Google Calendar and Slack Webhooks) once a candidate passes initial verification hurdles.

4.2 Use Case Specification

4.2.1 Apply CV

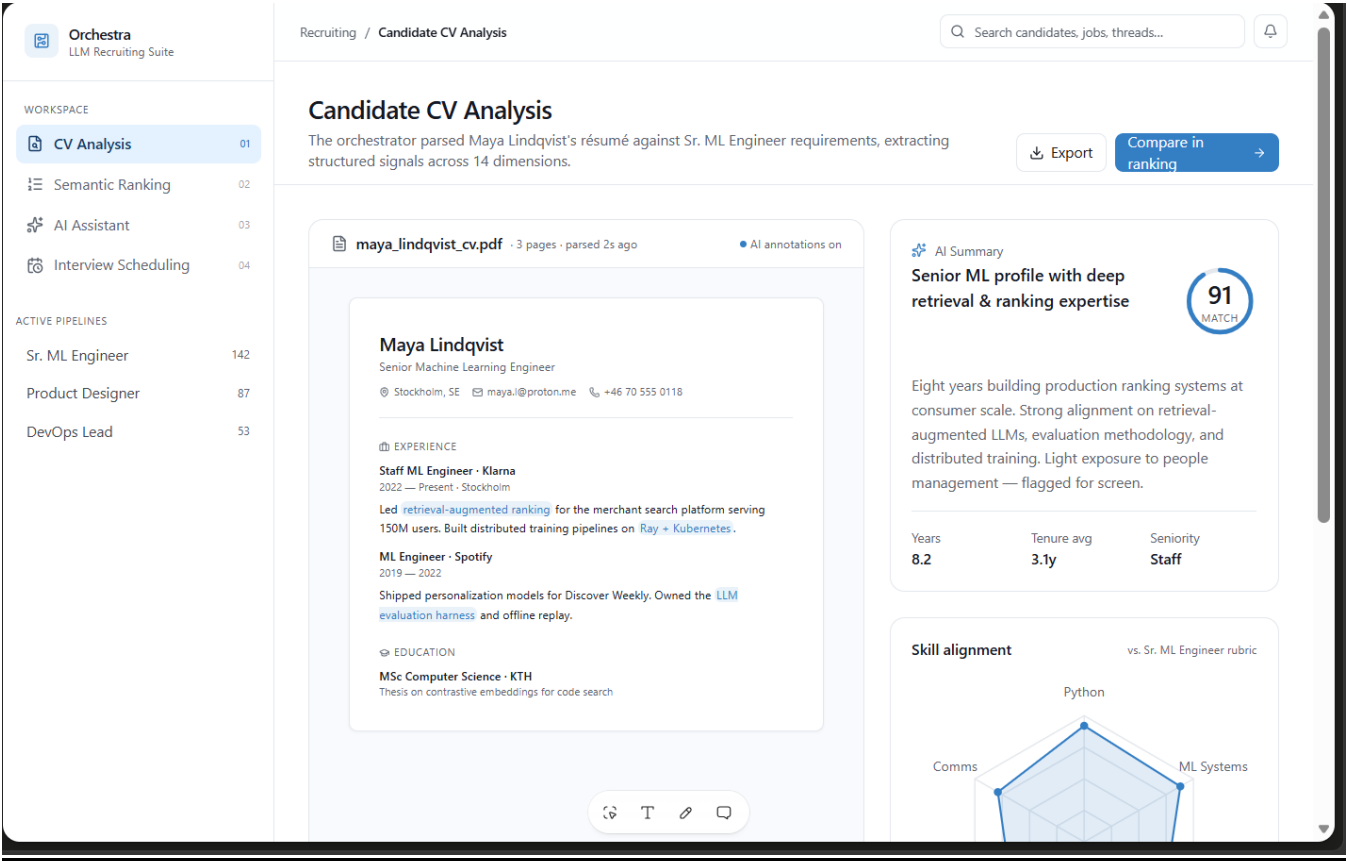
Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Use case ID	U001
Use Case	Apply CV
Brief Description	The Candidate uploads a CV to the recruitment system. The system automatically parses the CV, generates a structured summary using an LLM, and creates semantic embeddings for future candidate matching.
Actor	Candidate
Pre-Condition	1. Candidates can access the application. 2. CV document is available in PDF or DOCX format.
Result	1. CV is successfully stored. 2. Parsed candidate information is generated. 3. AI-generated summary is created. 4. Embedding vectors are stored for semantic search.

Main Scenario	1. Candidate selects Apply CV. 2. Candidate uploads a CV document. 3. The system validates the uploaded file. 4. The system parses the CV. 5. The system generates an AI summary. 6. The system creates semantic embeddings. 7. The system stores all processed information. 8. The system confirms successful submission.
Alternatice Scenarios	Alternative 1- Invalid File 1. Candidate uploads an unsupported file format. 2. The system displays an error message. Alternative 2 - Parsing Failure 1. CV parsing fails. 2. The system requests the candidate to upload another CV. Alternative 3 - AI Service Unavailable 1. Summary generation fails. 2. The system stores the raw CV and retries AI processing later.
Non-Functional Constraints	• CV processing should complete within a maximum of 1 minute. • The system should support PDF and DOCX. • Uploaded files must be securely stored. • AI processing should be fault tolerant.



4.2.2 Profile Ingestion & Split-Screen Verification Layout

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

Use case ID	U002
Use Case	Profile Ingestion & Split-Screen Verification Layout
Brief Description	Allows an authenticated HR Manager to upload raw candidate resume documents and instantly inspect AI-parsed visual analytical telemetry side-by-side with the original file layout
Actor	HR Manager, Autonomous AI Analytics Engine
Pre-Condition	1. The HR Manager must be securely logged into the SmartATS portal with appropriate access roles.

	The client browser environment must support persistent WebSocket channels and canvas streaming.
Result	- The raw PDF binary document is securely archived in cloud blob persistence storage. - An ingestion tracking message token is published to the queue cluster. - The active split-screen view updates with real-time parsed data metrics.
Main Scenario	- The HR Manager accesses the central talent acquisition pipeline workspace. - The HR Manager drags and drops a candidate's resume file (PDF layout) onto the file ingestion interface panel. - The system activates input filters to validate file format policies and size constraint parameters (under 10MB). - The system streams the verified binary asset to Microsoft Azure Blob Storage and captures a unique secure storage URL. - The system generates a candidate record token mapped to a new candidate UUID. - The system fires an asynchronous cv.received notification payload to the Azure Service Bus queue broker and pushes an HTTP 202 Accepted status token to the frontend client. - The interface dynamically triggers a Split-Screen Workspace UI container layout. - The system streams and displays the original candidate PDF on the left-hand viewport canvas panel. - The frontend application establishes a dedicated, listening WebSocket sub-channel handshake mapped to the candidate's UUID. - The system catches the payload frame and instantly visualizes the candidate profile metadata, interactive Radar Charts, and SVG career timelines on the right-hand layout panel.
Alternatice Scenarios	Alternatice 1 - Invalid File Format/Size Execution: At Step 3: If the file data fails policy validation checks (violates PDF MIME constraints or size boundary limits exceed 10MB): The system blocks further upload operations and cancels storage stream transactions.

	2. The system generates an HTTP 400 Bad Request client boundary code error. 3. The workspace interface catches the exception code and shows a validation failure banner alerting the user. 4. The layout returns safely to an idle ingestion holding state. Alternatice 2 - Asynchronous Engine Failure Timeout: At Step 10: If the background AI parsing worker nodes face deep server execution crashes or token timeout limits are reached: 1. The orchestration loop fails to update the processing state to completion. 2. The active WebSocket channel pushes an execution failure state token to the client interface. 3. The left-hand canvas retains the native PDF view, while the right-hand analytics block shifts into a fallback state UI. 4. The interface presents a manual data curation wizard option to the HR Manager to handle the exception gracefully.
Non-Functional Constraints	1. Rendering Deadline: The native client canvas must display page one of the file stream inside the left pane within 800ms of API confirmation. 2. Layout Integrity: The dual view pane must respond gracefully to panel resizing without triggering horizontal text layout overflow clipping.

The screenshot displays the SmartATS interface, which is used for managing the hiring process. The top navigation bar shows the current stage: Sourcing 142, Screening 38, Interview 12, and Offer 3. The main area is divided into two panels.

PDF Ingestion Stream: This panel on the left allows users to upload resumes. It features a large dashed box with a cloud icon and the text "Drag and drop engineering candidate PDF resume here to ingest". Below this, it specifies "Max 10 MB · PDF only" and provides a "Browse files" button.

Verification Workspace: This panel on the right shows the details for a specific candidate, Alex M. Richardson. It includes a "Verification Workspace" header with a "candidate-Bad313" link. The candidate's profile is displayed, including their name, title (Senior Backend Software Engineer), and contact information. The "EXPERIENCE" section lists three roles: Senior Backend Engineer at Nexus Systems Ltd. (Jan 2024 - Present), Backend Engineer at DataBridge Inc. (Mar 2022 - Dec 2023), and Junior Software Developer at TechStack Labs (Jun 2020 - Feb 2022). The "EDUCATION" section shows a B.Sc. in Computer Science from the University of Science (2014 - 2018). The "SKILLS" section lists various technologies and frameworks. The "CERTIFICATIONS" section lists AWS Solutions Architect - Pro, Certified Kubernetes Administrator, and HashiCorp Terraform Associate.

The right panel also features a "Verification Workspace" section with a "candidate-Bad313" link. It displays the candidate's overall affinity score of 87.4/100, which is high (91st percentile). The "OVERALL AFFINITY" section shows a match confidence of 87.4/100. The "TECHNICAL SKILL MATRIX" section shows a multi-axis competency chart with scores for Backend (88), Frontend (62), Cloud (75), InfoSec (54), and ML / AI (71). The "Career Trajectory" section shows a chronological professional milestones timeline, with the current role being Senior Backend Engineer at Nexus Systems Ltd. (Jan 2024 - Present).

4.2.3 Find Candidates

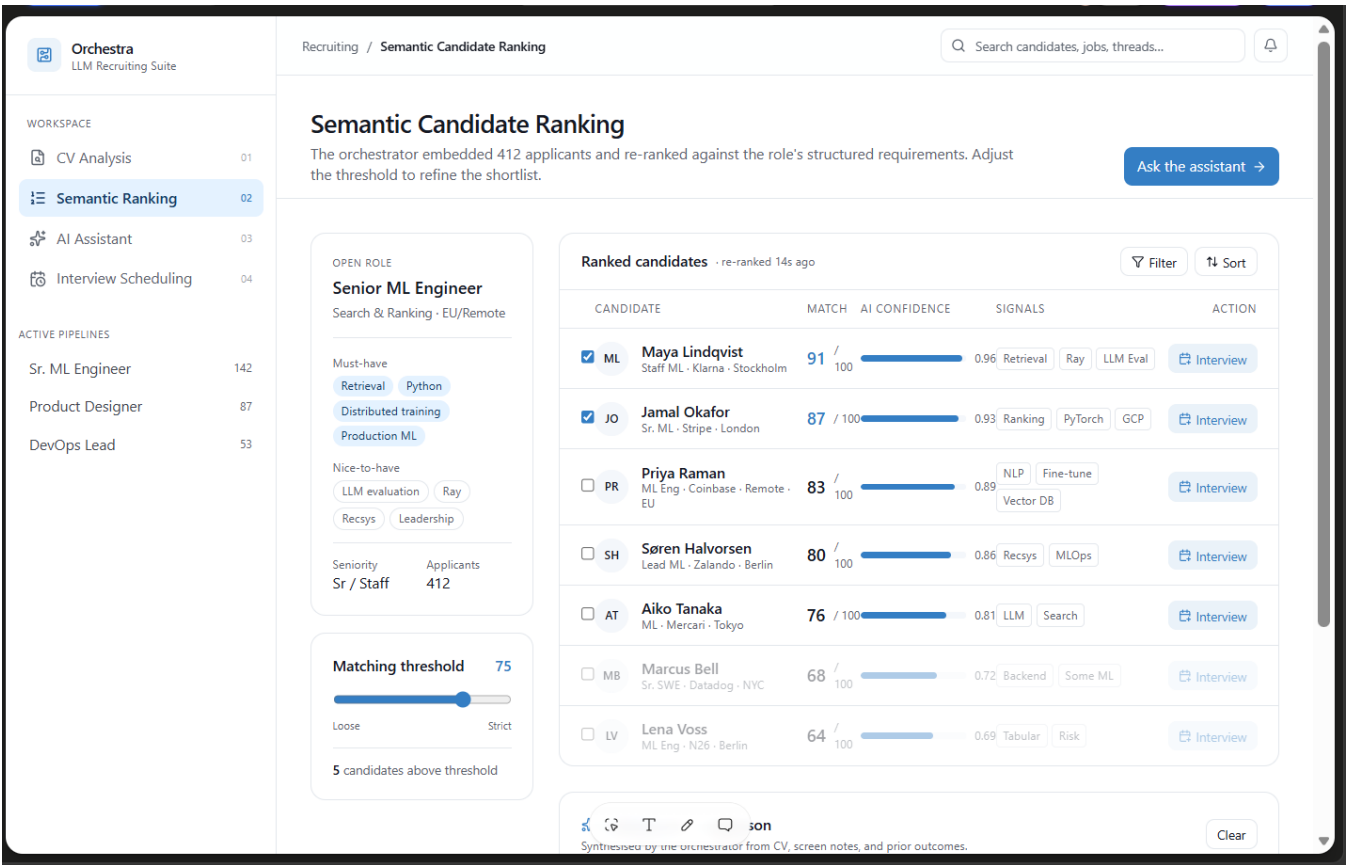
Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Use case ID	U003
Use Case	Find Candidates
Brief Description	HR searches candidates by entering job requirements. The system performs semantic ranking to retrieve the most relevant candidates.
Actor	HR / Tech Lead
Pre-Condition	1. Job Requirement exists. 2. Candidates database is available. 3. Candidate embeddings have been generated.
Result	A ranked list of matching candidates is displayed.
Main Scenario	1. HR selects Find Candidates. 2. HR enters job requirements. 3. The system performs Semantic Candidate Ranking. 4. The system calculates similarity scores. 5. Candidates are ranked. 6. HR reviews the ranked results. 7. HR selects candidates for interviews.
Alternatice Scenarios	Alternative - 1 1. No candidate satisfies the minimum similarity threshold. 2. The system suggests lowering the threshold. Alternative - 2 1. No embedding exists. 2. The system informs HR that candidate indexing is still in progress. Alternative - 3 1. Database connection fails. 2. The system displays an error message.

Non-Functional Constraints	Similarity search must use semantic embeddings.



4.2.4 Cross-Channel Profile Enrichment & Manual Sync Trigger

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127337 - Nguyễn Tiến Cường

Use case ID	U004
Use Case	Cross-Channel Profile Enrichment & Manual Sync Trigger

Brief Description	<i>Allows an authorized HR Manager to manually trigger data harvesting from external third-party APIs (GitHub, LinkedIn) based on the candidate's extracted profile details and seamlessly integrate enriched technical telemetry into the analytics dashboard.</i>
Actor	<i>HR Manager, Portfolio-Enrich Engine (External Third-Party System), Autonomous AI Analytics Engine.</i>
Pre-Condition	<i>1. The HR Manager must be authenticated and actively viewing the candidate's initial parsed profile page.</i> <i>2. The candidate's raw resume data must already be processed by the Ingestion Gateway, with a valid candidate_uuid generated.</i> <i>3. External third-party identity APIs (GitHub API, LinkedIn API) must be operational with authenticated credential handshakes configured.</i>
Result	<i>Verified external portfolio metrics and professional history are safely integrated into the relational vector database without blocking the client UI thread, updating the centralized visual dashboards dynamically.</i>

Main Scenario

1. The HR Manager accesses the specific candidate profile view displaying raw PDF analytics.
2. The HR Manager clicks the "Run Sync" action button on the top navigation bar to initialize external verification.
3. The system captures the request and securely extracts professional social identifiers (e.g., GitHub username, LinkedIn profile URL) from the candidate's indexed database record.
4. The platform launches an asynchronous background worker task, dispatching secure payload queries to the GitHub API and LinkedIn API.
5. The Portfolio-Enrich module crawls and analyzes public code repositories (parsing README.md files) from GitHub and structured employment history timelines from LinkedIn.
6. The background pipeline normalizes the fetched unstructured metadata into a unified technical capability schema.
7. The database synchronization layer executes vector transformations, updating and enriching candidate skill attributes within the pgvector/PostgreSQL cluster.
8. The system signals the Autonomous AI Analytics Engine to recalculate overall match matrices and instantly pushes updated telemetry frames across the active WebSocket tunnel.
9. The UI refreshes the Left Viewport to present the scrollable external integration metrics and projects

enhanced Canvas Radar Charts and SVG timelines on the Right Panel.

Alternatice Scenarios*Alternative 1 - Missing External Profile Identifiers:*

At Step 3: If the candidate's raw resume does not contain any recognizable GitHub link or LinkedIn profile pointers:

- *The system immediately flags the synchronization transaction status as "No External Profiles Identified".*
- *The workspace interface intercepts the status token and displays a warning banner to the user.*
- *The workflow terminates safely, keeping the baseline CV analytics unchanged.*

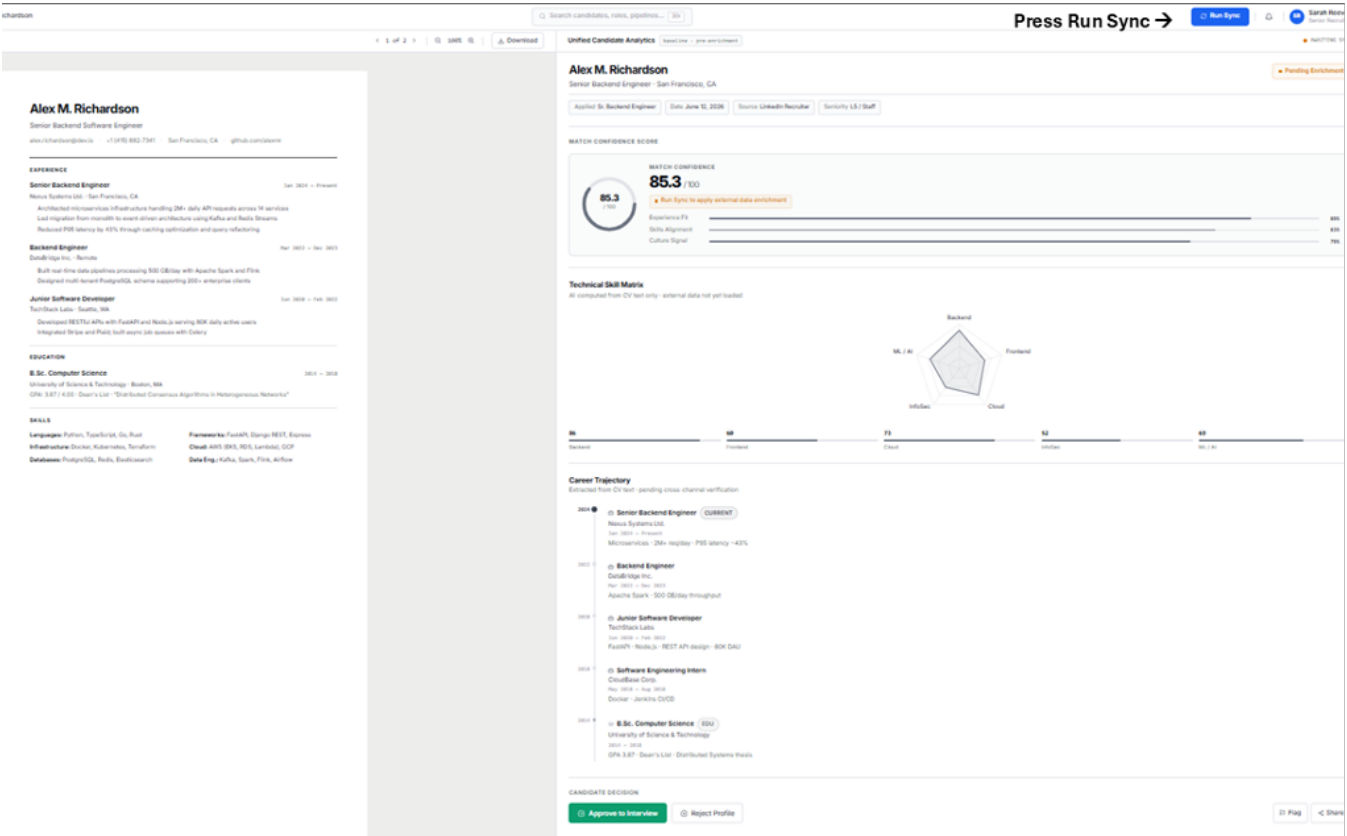
Alternative 2 - Third-Party API Interruption or Rate Limit Breaches:

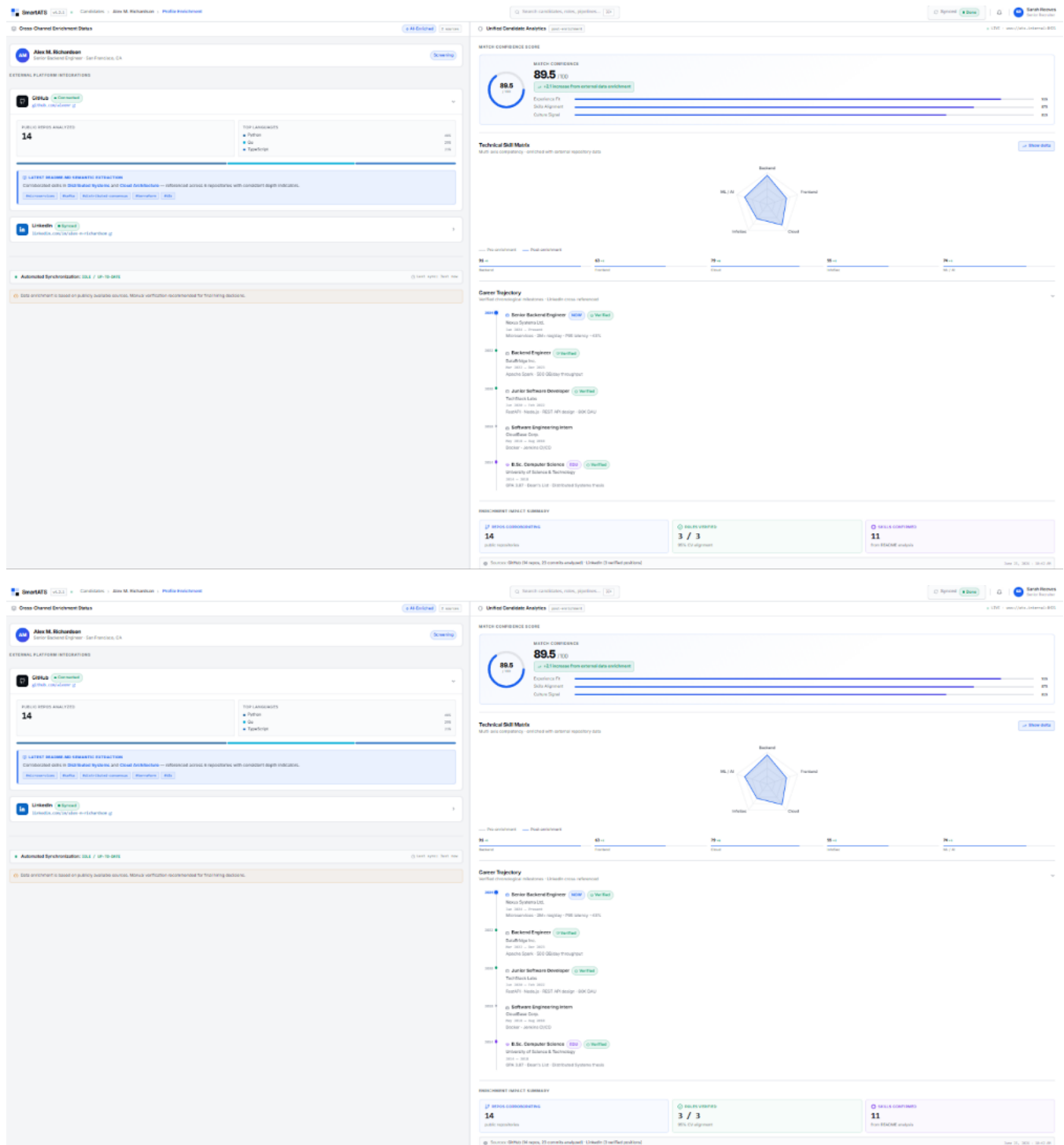
At Step 4: If the third-party integrations experience server-side crashes or return HTTP 429 (Too Many Requests) / Timeout errors:

- *The platform registers the execution exception inside the logging queue backbone.*
- *The background task engine initiates an automated background retry routine utilizing an Exponential Backoff strategy.*
- *If errors persist beyond the hardcoded maximum threshold, the sync state flags "Enrichment Failed" via WebSocket, maintaining the original raw CV baseline while providing a manual retry control option.*

Non-Functional Constraints

- Asynchronous Compliance: The external data harvesting pipeline must execute strictly as a non-blocking background thread, preserving interactive interface responsiveness below a 200ms threshold.
- Rate Limiting Guardrails: The integration layer must enforce internal throttling matrices to comply with third-party endpoint rate-limiting policies, preventing API token revocations or temporary IP blocking.





Click this [link](#) for a clearer view of the Figma prototype.

4.2.5 Chat AI Assistant

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

Use case ID	U005
Use Case	Chat AI Assistant
Brief Description	Users interact with the AI Assistant using natural language. The Autonomous LLM Orchestrator analyzes user intent and invokes appropriate tools such as Semantic Candidate Ranking or Interview Scheduling.
Actor	HR / Tech Lead
Pre-Condition	The user has access to the chatbot. AI services are available.
Result	The requested task is completed and the AI returns an appropriate response.
Main Scenario	1. Users submit a natural language request. 2. AI Workflow Orchestration analyzes the intent. 3. The system selects the appropriate tool. 4. The selected tool is executed. 5. AI summarizes the result. 6. Response is returned to the user.
Alternative Scenarios	Alternative - 1 1. The AI cannot determine the user's intent. 2. The system asks for clarification. Alternative - 2 1. Required tools are unavailable.

	- The system informs the user that the request cannot be completed. Alternative - 3 - Execution timeout. - The system retries or returns a failure message.
Non-Functional Constraints	- AI response time should be less than 1 minutes. - Tool execution should be modular. - The Orchestrator should support future tool extensions.

The screenshot displays the Orchestra LLM Recruiting Suite interface. The main workspace is titled "AI Recruitment Assistant" and shows a workflow execution for a "Sr. ML Engineer" pipeline. The workflow consists of three steps: "semantic_rank" (DONE), "calendar.find_slots" (DONE), and "outreach.draft" (AWAITING). A "TOP 5 SHORTLIST" is shown with Maya Lindqvist as the top match. The interface also includes a "Tool orchestration" panel on the right with various tools like "semantic_rank", "ats.query", "calendar.find_slots", "outreach.draft", and "background.check". A "Suggested actions" panel at the bottom right offers actions like "Trigger reference checks for Maya & Jamal" and "Re-rank pipeline now that 32 new CVs arrived".

4.2.6 ABAC Security Data Masking

Written by: 24127337 - Nguyễn Tiến Cường

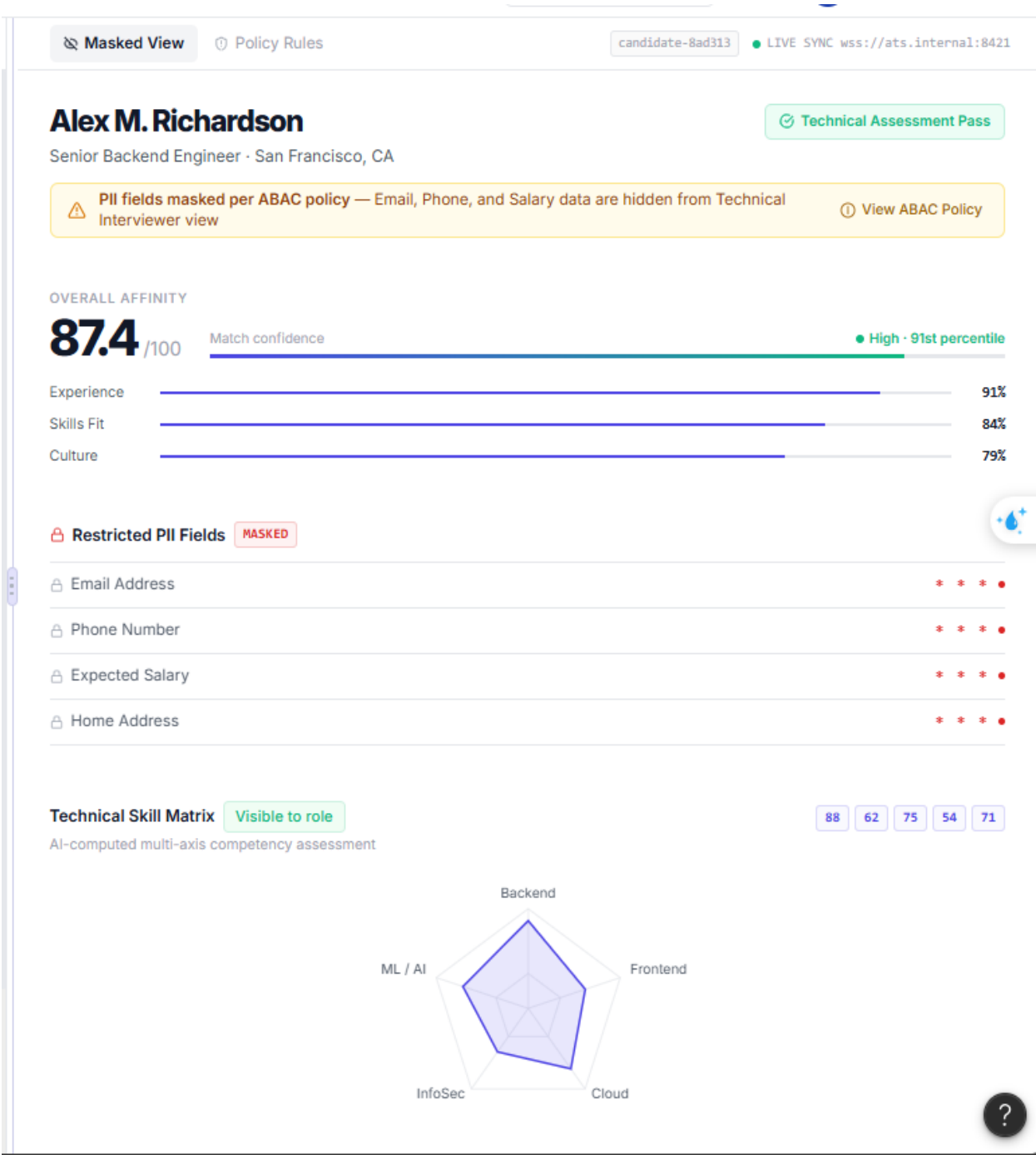
Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127382 - Trần Nhật Hoàng

Use case ID	U006
Use Case	<i>ABAC Security Data Masking</i>
Brief Description	Allows a Technical Interviewer to review a candidate's full analytical profile (Radar Chart, career timeline, skill breakdown) while the system automatically enforces Attribute-Based Access Control (ABAC) by recursively masking sensitive Personally Identifiable Information (PII) fields (email, phone, expected_salary) to protect data privacy and eliminate unconscious bias.
Actor	<i>Technical Interviewer, ABAC Policy Engine (System)</i>
Pre-Condition	1. The Technical Interviewer must be authenticated via JWT with the role 'interviewer'. 2. A candidate profile must already exist in the system (ingested and parsed by the AI Core). 3. The candidate record contains PII fields (email, phone_number, expected_salary) that are unmasked for HR roles. 4. The ABAC policy configuration file defines masking rules for the 'interviewer' role.
Result	1. The Technical Interviewer sees the candidate's professional analytics with all PII fields masked as '***'. 2. The original unmasked data remains intact in the database. 3. An audit log entry is recorded with user ID, role, candidate UUID, and timestamp.
Main Scenario	1. The Technical Interviewer navigates to a specific candidate's profile through the Workspace UI.

	2. The frontend sends a GET /api/candidates/{candidate_uuid} request with the interviewer's JWT token. 3. The API Gateway decodes the JWT and extracts the role claim ('interviewer'). 4. The Backend invokes the ABAC Policy Engine before returning the response. 5. The ABAC Policy Engine reads the masking rules from the policy configuration. 6. For each field defined in the masking rules (email, phone_number, expected_salary), the engine applies a recursive masking function that replaces the original value with '***'. 7. The masked JSON payload is returned to the frontend. 8. The UI renders the candidate analytics (Radar Chart, timeline, skill scores) with PII fields displayed as '***'.
Alternative Scenarios	Alternative 1 — Missing or Expired JWT Token: • <i>At Step 2–3:</i> If the JWT token is missing or expired, the API Gateway returns HTTP 401 Unauthorized. The frontend redirects the user to the login page. Alternative 2 — Role Not Recognized by ABAC Policy: • <i>At Step 4–5:</i> If the user's role is not defined in the ABAC policy configuration, the Policy Engine defaults to the most restrictive masking level (mask ALL PII fields). An audit warning is logged for the System Administrator. Alternative 3 — Candidate Record Has No PII Fields: • <i>At Step 6:</i> If the candidate record has sparse or missing PII fields, the masking function returns an empty string for missing fields. No error is raised.

Non-Functional Constraints	• Response Time: The ABAC Policy Engine MUST complete masking within 150ms. • Security Coverage: 100% of defined PII fields MUST be masked automatically. • Audit Compliance: Every access MUST be logged with user, role, and timestamp.
----------------------------	--



4.2.7 Contextual Candidate Interview Scheduling

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127254 - Hồ Đình Trí

Use case ID	U007
Use Case	<i>Contextual Candidate Interview Scheduling</i>
Brief Description	Allows an HR Manager to automatically generate interview time slots by scanning the Google Calendar free/busy data of multiple Tech Leads, computing common overlaps using a Sweep-Line algorithm, and presenting only slots ≥ 45 minutes for confirmation.
Actor	<i>HR Manager, Google Calendar API</i> (External System), <i>SmartATS Scheduling Engine</i> (System)
Pre-Condition	1. The HR Manager must be authenticated with the role 'recruiter' or 'admin'. 2. A candidate must have passed the initial screening (assessment status = 'Technical Assessment Pass'). 3. At least two Tech Lead interviewers must be assigned to the candidate's interview panel. 4. Google Calendar API credentials (OAuth 2.0 service account) must be configured and valid. 5. Each Tech Lead's primary calendar ID must be stored in the system.
Result	1. A Google Calendar event is created with all participants invited. 2. A Slack notification is dispatched to the configured HR channel. 3. The candidate's status is updated to 'Interview Scheduled' in the database. 4. An audit log entry is recorded with scheduling details.
Main Scenario	1. The HR Manager navigates to the candidate's profile and clicks 'Schedule Interview'.

	2. The frontend sends GET /api/interviews/slots?candidate_uuid={uuid} to the backend. 3. The Scheduling Engine fetches the list of assigned Tech Lead interviewers and their Google Calendar IDs from the database. 4. For each Tech Lead, the engine calls the Google Calendar API free/busy endpoint (timeMin=now, timeMax=+14 days). 5. The engine gathers all free/busy intervals and applies a Sweep-Line algorithm: - Sort all interval start/end events by time. - Sweep through the timeline, maintaining a running count of available interviewers. - Identify time intervals where the count equals the total number of interviewers (full overlap). - Filter intervals with minimum duration of 45 minutes 6. The ordered list of valid slots is returned to the frontend and rendered in a time-picker UI. 7. The HR Manager selects a preferred slot and clicks 'Confirm'. 8. The system creates a Google Calendar event with candidate and interviewers as attendees, then posts a notification to the Slack Webhook.
Alternative Scenarios	Alternative 1 — Google Calendar API Disconnected / Token Expired: <i>At Step 4:</i> If the Google Calendar API returns 401 or the connection fails, the engine retries once with a refreshed OAuth token. If the second attempt also fails, the system

	returns an error: 'Calendar service unavailable. Contact administrator.' Alternative 2 — No Common Free Slot Found: • <i>At Step 5c:</i> If the Sweep-Line algorithm finds zero overlapping intervals, the system returns an empty slot list with the message: 'No common free slots found in the next 14 days.' Alternative 3 — All Overlapping Slots Below 45 Minutes: • <i>At Step 5d:</i> If all overlaps are shorter than 45 minutes, the system returns the longest available slot with a warning message.
Non-Functional Constraints	• Query Deadline: Calendar free/busy queries for up to 5 Tech Leads MUST complete within 3 seconds. • Precision: Sweep-Line algorithm MUST guarantee zero false-positive overlaps. • Availability: If Calendar API is unreachable, fallback to cached data with a warning banner.

SmartATS [v4.2.1]

Q Search candidates...

Screening 24 Technical 8 Final 3

HR Manager

Candidates > Alex M. Richardson > Schedule Interview

Alex M. Richardson

Senior Backend Engineer

Technical Assessment Pass

Applied

Sr. Backend Engineer - June 12, 2026

Source

LinkedIn Recruiter

Seniority

LS / Staff

Department

Platform Engineering

UUID

candidate-8ad313

Affinity Score

87.4 / 100

INTERVIEW PANEL (3)

SC Sarah Chen

Staff Engineer

Connected

MP Mike Park

Tech Lead

Connected

LW Lisa Wong

Engineering Manager

Sync Required

Available Interview Slots

Common free slots ≥45 min — computed via Sweep-Line algorithm

Sweep-Line Algorithm Visualization

RECOMMENDED

Wednesday, July 15, 2026

10:00 AM — 11:00 AM (60 min)

3/3 available

High availability — all panelists confirmed

Thursday, July 16, 2026

2:00 PM — 2:45 PM (45 min)

3/3 available

Minimum duration slot

No more common slots found for the next 14 days

July 2026

Su	Mo	Tu	We	Th	Fr	Sa
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Available Unavailable

Confirm Wednesday, July 15 — 10:00 AM

Send Calendar Invites

Slack #hiring-alerts

Auto-notify

Interview scheduled: Alex M. Richardson with Sarah Chen, Mike Park, Lisa Wong on Jul 15 at 10:00 AM

Google Calendar API rate limit approaching — 62/100 requests used today

Retry

SESSION STATS

Days Scanned

14

Windows Found

4

Slots ≥45min

?

Panelists

3/3

HCMUS | SE Dept.

47



Interview Scheduled!

Alex M. Richardson
Wednesday, July 15 - 10:00 AM - 11:00 AM

Panelists notified

SC

 Sarah Chen

MP

 Mike Park

LW

 Lisa Wong

Send Google Calendar Invite

Notify via Slack

[Return to Candidate Profile](#)



No Common Free Slots Found

Sweep-Line scanned 14 days across 3 interviewers. No overlapping window ≥ 45 minutes exists.

Notify Panelists to Update Calendar

Adjust Duration

[Back](#)

Google Calendar API disconnected. Auto-scheduling unavailable.

Enter Available Times Manually

Add availability for each panelist below.

SC	Sarah Chen	10:00 AM	—	11:00 AM
MP	Mike Park	10:00 AM	—	11:00 AM
LW	Lisa Wong	10:00 AM	—	11:00 AM

Save Manual Slots

Retry in 30s

5

AI Usage Declaration

Team Declaration on AI Tool Utilization

In accordance with the course guidelines and academic integrity policies, our team has integrated Artificial Intelligence (AI) tools as supportive collaborators throughout the Requirements Analysis phase of the SmartATS project. AI assistance was utilized strictly for brainstorming architectural patterns, refining technical English terminology, and generating syntactic baselines for system modeling. The core domain logic, specific system constraints, and functional specifications remain entirely the original work of the team members.

5.1 Generative AI Tools Leveraged

- **Google Gemini & OpenAI ChatGPT:** Utilized primarily as conversational research assistants to discover best practices regarding Attribute-Based Access Control (ABAC) and to refine the mathematical phrasing of the Sweep-Line overlap detection algorithm.
- **Mermaid.js Live Editor AI:** Used to generate the initial structural syntax for the horizontal Use Case Model boundary, which was subsequently manual-tuned and customized by the team to eliminate overlapping paths.

5.2 Scope of AI Assistance

- **Documentation Refinement:** Generative models were used to audit our raw functional requirement descriptions, transforming informal notes into formal, engineering-focused specifications compliant with IEEE standards.
- **Alternative Scenario Brainstorming:** AI was prompted to suggest potential edge cases for third-party API failures (such as GitHub rate limits or Google Calendar credential expirations), which helped populate our alternative use case kịch bản.

5.3 Human Validation and Ownership

All AI-generated suggestions underwent rigorous peer review and verification by the assigned task owners on Jira. Any pseudocode or architectural concept provided by AI was manually adapted, debugged, and verified against our local project scope to guarantee data consistency and system feasibility.

6

Presentation

Link video on Youtube :<https://youtu.be/T5Z8DjrvSUw>

This video records our official team sync and presentation covering the Requirements Analysis phase for the SmartATS platform—an AI-powered Applicant Tracking System utilizing Large Language Models (LLMs) and Semantic Search to revolutionize modern corporate recruitment.

In this session, our team walks through the finalized Software Requirements Specification (SRS) document, demonstrating the tight synchronization between our Jira task management board and the architectural system artifacts.

Timestamps & Speaker Lineup:

- **00:00 - Introduction & Project Vision** (Presented by the Team)
- **00:45 - Problem Statement & Proposed Solution Architecture** (Presented by Tran Nhat Hoang - 24127382)
- **02:15 - Ingestion Infrastructure & Verification Workspace Layout** (Presented by Ho Dinh Tri - 24127254)
- **03:45 - Cross-Channel Profile Enrichment & External API Core** (Presented by Nguyen Khanh Toan - 24127252)
- **05:15 - Enterprise ABAC Security, PII Masking & Interview Scheduling** (Presented by Nguyen Tien Cuong - 24127337)
- **06:45 - Team Conclusion & Next Engineering Milestones** (Presented by the Team)

Core Tech Stack Highlighted: FastAPI (Python), React.js (TypeScript), PostgreSQL with pgvector, Microsoft Azure Blob Storage & Service Bus, Model Context Protocol (MCP) SDK.

All tasks discussed have been successfully tracked, reviewed, and updated on Jira to maintain 100% accountability and engineering compliance.

7

Reflective Report

7.1 Most Helpful Sections for System Implementation

Through the execution of this Requirement Analysis milestone, our team identified specific sections within the template that offer immense practical value for our upcoming development cycles:

- **Section 3.2 (Functional Requirements Specification) & Section 4.2 (Use Case Specifications):** These components serve as the definitive blueprint for our backend endpoint routing and state-machine logic. For instance, the precise step-by-step mapping of U002 (Profile Ingestion) and U007 (Interview Scheduling) directly informs how our FastAPI gateways must handle multipart form-data, cloud stream transactions, and asynchronous message broker payloads via Azure Service Bus.
- **Section 3.3 (Non-Functional Requirements Specification):** Establishing rigid, quantitative constraints—such as the 150ms execution deadline for recursive ABAC masking and the 800ms low-latency PDF rendering threshold—is critical. It shifts our development focus from basic functionality to precise performance indexing, forcing us to incorporate Redis caching and optimized database queries early in the architecture phase.
- **System UI Prototypes / Mockups:** Having a high-fidelity visual layout linked directly to each use case prevents feature creep and aligns frontend React.js component development with backend data schemas, ensuring the team works toward a unified system state.

7.2 Less Critical or Redundant Sections

While the template provides a comprehensive framework, certain elements introduced minor redundancies that could be streamlined in future iterations:

- **Overlap Between General Functional Descriptions and Use Case Scenarios:** We found that describing system actions in natural language within Section 3.2 often repeated the detailed operational steps formalized inside the Main Scenarios of Section 4.2. In a fast-paced development workflow, merging these into a single, comprehensive Use Case Specification block would eliminate duplication and reduce documentation overhead.
- **Early High-Level System Context vs. Detailed Architecture Mapping:** The abstract System Context Diagram provides a good initial overview, but its value decreases rapidly once the detailed, granular Use Case Models and module boundaries are established. For engineering execution, the precise boundary definitions in Section 4 are far more actionable than the high-level generalizations in Section 3.

7.3 Team Conclusion

Overall, this documentation process has forced our team to transition from abstract ideas into structured, platform-level technical thinking. It has effectively bridged the gap between general recruitment problems and concrete, cloud-native engineering solutions, giving us a solid, validated baseline to start coding immediately.